

# **Informe de Mejoras y Correcciones - AA1**

Gestión de Cine y Base de Datos

Sandro López Díaz

**Base de Datos y Programación**

**26/01/2026**

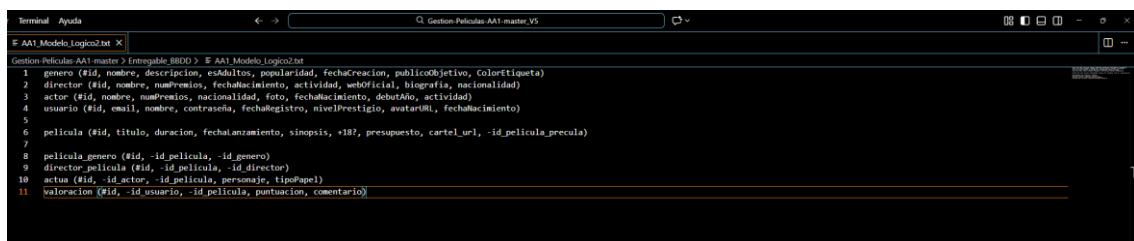


Tras la defensa del proyecto, he realizado una revisión profunda para subsanar los errores detectados y completar los requisitos pendientes. Este documento detalla las correcciones técnicas y lógicas implementadas tanto en la base de datos como en la aplicación JavaFX.

# Lógica de Base de Datos

**Corrección del Diseño Lógico (Sintaxis y Relaciones)** Uno de los puntos críticos fue la discrepancia entre el diagrama visual (yEd) y la definición textual de la lógica. He reescrito la estructura para cumplir estrictamente con las reglas de integridad referencial, asegurando que las relaciones **Muchos a Muchos (N:M)** tengan sus propias tablas intermedias con IDs propios, tal y como se refleja en el diagrama.

- **Lo que he mejorado:** He añadido las tablas `director_pelicula` y `pelicula_genero`, y he corregido la notación de claves foráneas (`-id_...`) para que coincida con la sintaxis del diagrama.



```
1 genero (#id, nombre, descripcion, esAdulto, popularidad, fechaCreacion, publicoObjetivo, ColorEtiqueta)
2 director (#id, nombre, numPremios, fechaNacimiento, actividad, webOficial, biografia, nacionalidad)
3 actor (#id, nombre, numPremios, nacionalidad, foto, fechaNacimiento, debutAño, actividad)
4 usuario (#id, email, nombre, contraseña, fechaRegistro, nivelPrestigio, avatarUrl, fechaNacimiento)
5
6 pelicula (#id, titulo, duracion, fechaLanzamiento, sinopsis, *18*, presupuesto, cartel_url, -id_pelicula_preced)
7
8 pelicula_genero (#id, -id_pelicula, -id_genero)
9 director_pelicula (#id, -id_pelicula, -id_director)
10 actor_pelicula (#id, -id_pelicula, -id_actor, personaje, tipoPapel)
11 valoracion (#id, -id_usuario, -id_pelicula, puntuacion, comentario)
```

## Integridad y SQL (Lo que faltaba)

Para cumplir con los requisitos de negocio y asegurar la integridad de los datos, he implementado una estructura robusta en MariaDB compuesta por 5 Vistas estratégicas y un sistema de Triggers híbrido (Automatización + Seguridad).

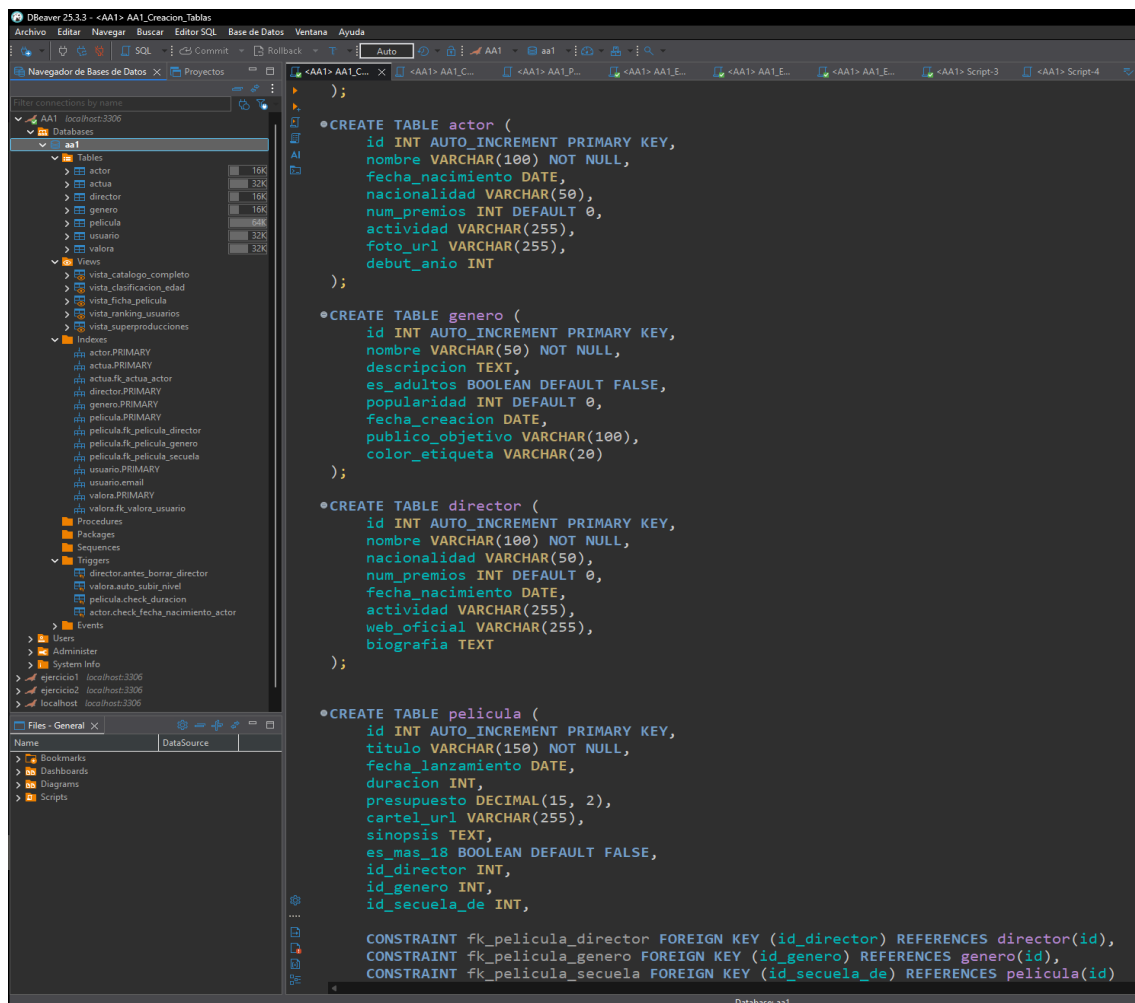
**A. Vistas Implementadas (Reporting)** Se han creado 5 vistas para simplificar el acceso a datos complejos desde la aplicación Java:

1. **vista\_catalogo\_completo:** Unificación de películas con sus directores y datos principales.
2. **vista\_clasificacion\_edad:** Filtrado rápido para control parental y segmentación de audiencia.
3. **vista\_ficha\_pelicula:** Vista detallada para la generación de informes individuales.
4. **vista\_ranking\_usuarios:** Monitorización de la actividad y nivel de los usuarios.

## 5. vista\_superproducciones: Filtrado financiero de proyectos de alto presupuesto.

B. Triggers: Se ha reforzado la lógica de negocio con 4 Triggers divididos en dos áreas:

- Automatización:
  - **valora\_auto\_subir\_nivel**: Gamificación automática. Al valorar películas, el usuario sube de nivel dinámicamente.
  - **director\_antes\_borrar**: Gestión de integridad referencial en cascada personalizada antes de eliminar registros.
- Integridad de Datos (Nuevos):
  - **check\_duracion**: Impide la inserción de películas con duración errónea ( $\leq 0$  minutos).
  - **check\_fecha\_nacimiento**: Evita inconsistencias temporales (actores nacidos en el futuro).



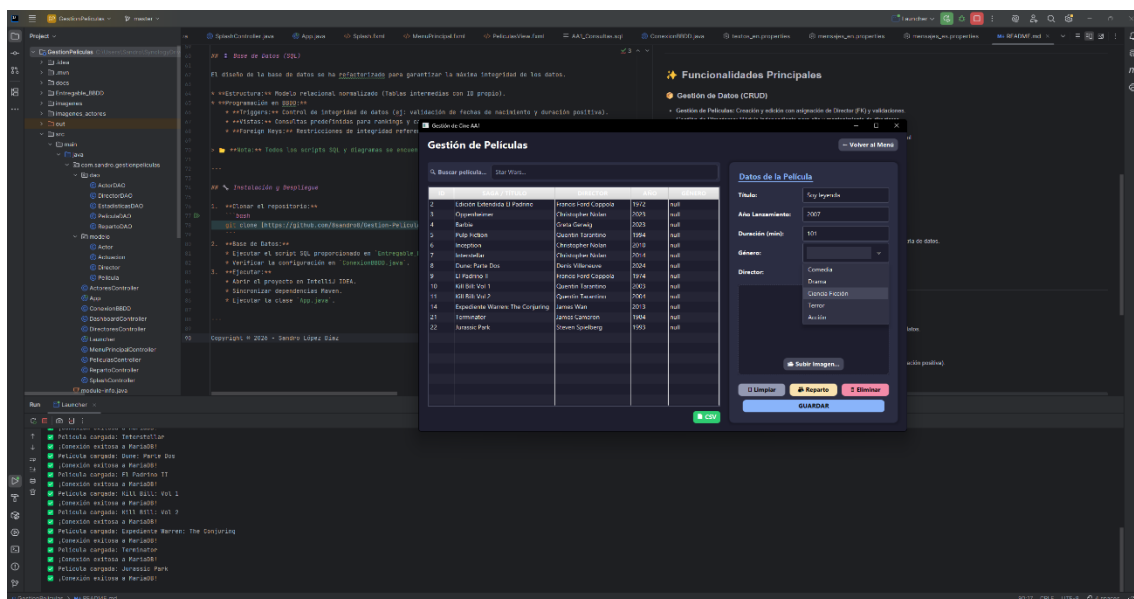
```
CREATE TABLE actor (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    fecha_nacimiento DATE,  
    nacionalidad VARCHAR(50),  
    num_premios INT DEFAULT 0,  
    actividad VARCHAR(255),  
    foto_url VARCHAR(255),  
    debut_anio INT  
);  
  
CREATE TABLE genero (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    descripcion TEXT,  
    es_adultos BOOLEAN DEFAULT FALSE,  
    popularidad INT DEFAULT 0,  
    fecha_creacion DATE,  
    publico_objetivo VARCHAR(100),  
    color_etiqueta VARCHAR(20)  
);  
  
CREATE TABLE director (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(100) NOT NULL,  
    nacionalidad VARCHAR(50),  
    num_premios INT DEFAULT 0,  
    fecha_nacimiento DATE,  
    actividad VARCHAR(255),  
    web_oficial VARCHAR(255),  
    biografia TEXT  
);  
  
CREATE TABLE pelicula (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    titulo VARCHAR(150) NOT NULL,  
    fecha_lanzamiento DATE,  
    duracion INT,  
    presupuesto DECIMAL(15, 2),  
    cartel_url VARCHAR(255),  
    sinopsis TEXT,  
    es_mas_18 BOOLEAN DEFAULT FALSE,  
    id_director INT,  
    id_genero INT,  
    id_secuela_de INT,  
  
    CONSTRAINT fk_pelicula_director FOREIGN KEY (id_director) REFERENCES director(id),  
    CONSTRAINT fk_pelicula_genero FOREIGN KEY (id_genero) REFERENCES genero(id),  
    CONSTRAINT fk_pelicula_secuela FOREIGN KEY (id_secuela_de) REFERENCES pelicula(id)
```

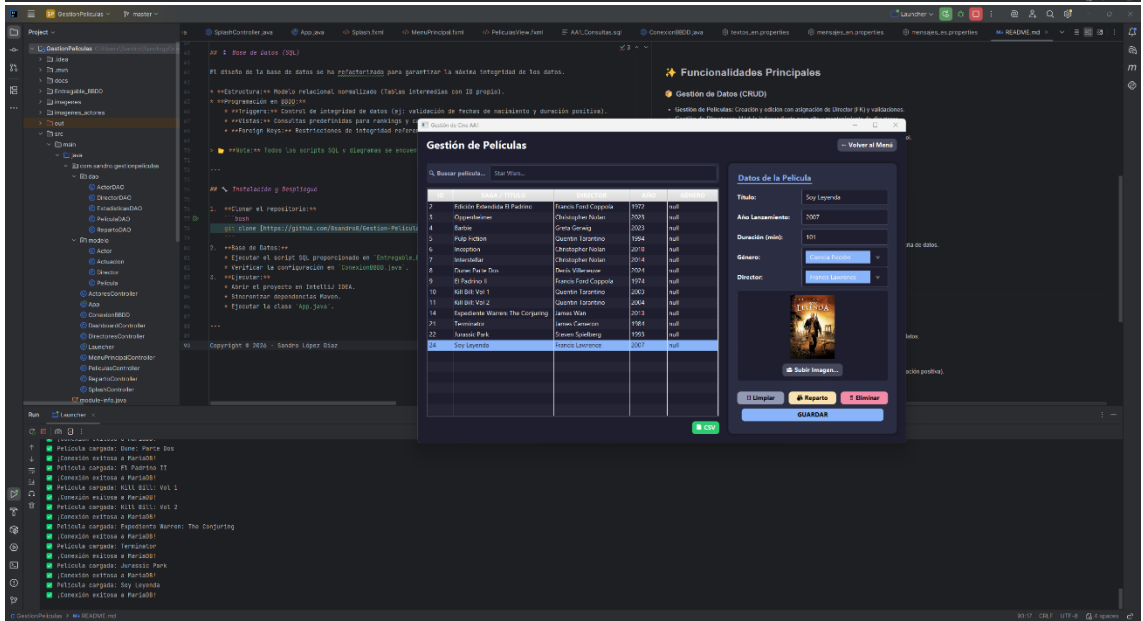
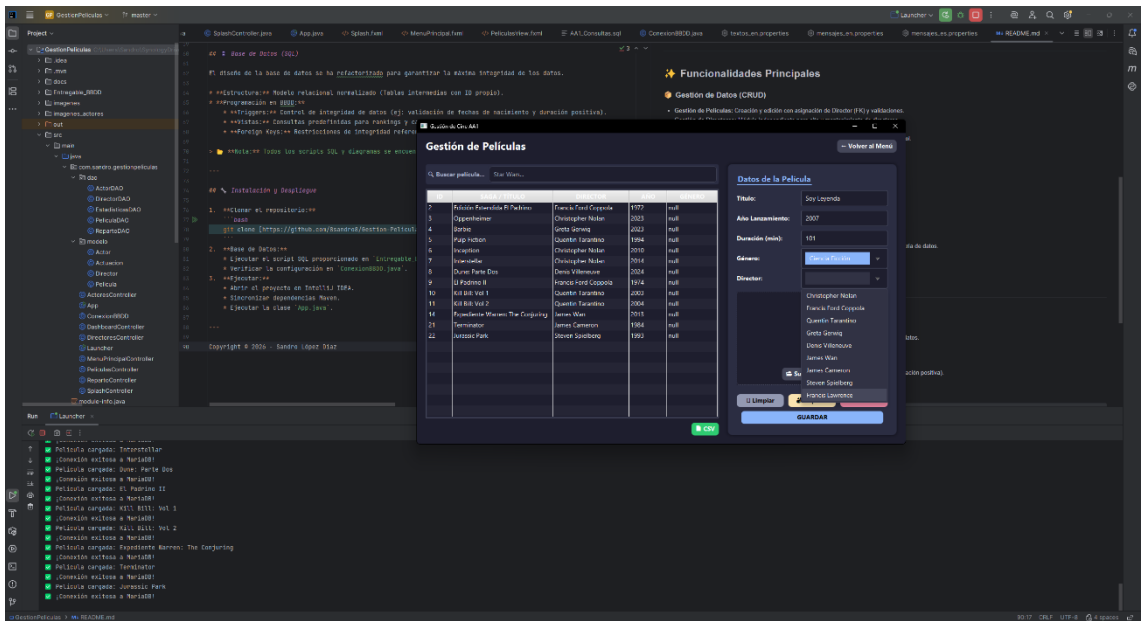
## La Aplicación (Solución del Crash y Mejoras UI)

Solución al Error de Creación y Mejora de Usabilidad: Durante la defensa, la aplicación falló al intentar guardar una nueva película porque el sistema intentaba procesar el nombre del género como texto, en lugar de enlazarlo correctamente con su ID.

Para solucionar esto de raíz y mejorar la experiencia del usuario, he realizado dos cambios clave en el controlador `PeliculasController`:

- **Implementación de ComboBox:** He sustituido los campos de texto libre por desplegables (ComboBox) tanto para **Director** como para **Género**.
  - *Ventaja:* El usuario ahora selecciona opciones reales que ya existen en la base de datos (por ejemplo, selecciona "Nolan" de la lista).
  - *Seguridad:* Esto evita errores tipográficos y asegura que el programa envíe siempre un **ID válido** a MariaDB, eliminando por completo el riesgo de fallo al guardar.
- **Corrección Lógica:** Internamente, el sistema ahora captura el objeto seleccionado en el desplegable y extrae su ID para la sentencia SQL, garantizando la integridad de la relación.





## Nuevas Funcionalidades y Separación de Responsabilidades

Gestión Independiente de Directores (Refactorización) Atendiendo a las correcciones sobre la mezcla de conceptos, he separado totalmente la lógica de Directores de la de Actores. Antes compartían estructura, lo que generaba confusión.

- **Módulo Propio:** He creado una pantalla exclusiva de "Gestión de Directores". Ahora funcionan de forma independiente.
- **Datos Específicos:** Al tener su propia entidad, he podido añadir campos que antes no tenían sentido para un actor, como "Web Oficial" o "Años en activo", enriqueciendo la base de datos.
- **Integración:** Estos directores creados aquí son los que luego alimentan automáticamente el ComboBox de la pantalla de películas.

The screenshot shows a web application titled "Gestión de Cine AA1". The main section is "Gestión de Directores", which includes a search bar, a table of directors, and a form for adding or editing a director.

**Search Bar:** "Buscar: Buscar director por nombre..."

**Table of Directors:**

| ID | Nombre del Director  | Nacionalidad   |
|----|----------------------|----------------|
| 13 | Christopher Nolan    | Británico      |
| 14 | Francis Ford Coppola | Estadounidense |
| 15 | Quentin Tarantino    | Estadounidense |
| 16 | Greta Gerwig         | Estadounidense |
| 17 | Denis Villeneuve     | Canadiense     |
| 18 | James Wan            | Australiano    |
| 19 | James Cameron        | Canadiense     |
| 20 | Steven Spielberg     | Estadounidense |
| 21 | Francis Lawrence     | Austria        |

**Datos del Director Form:**

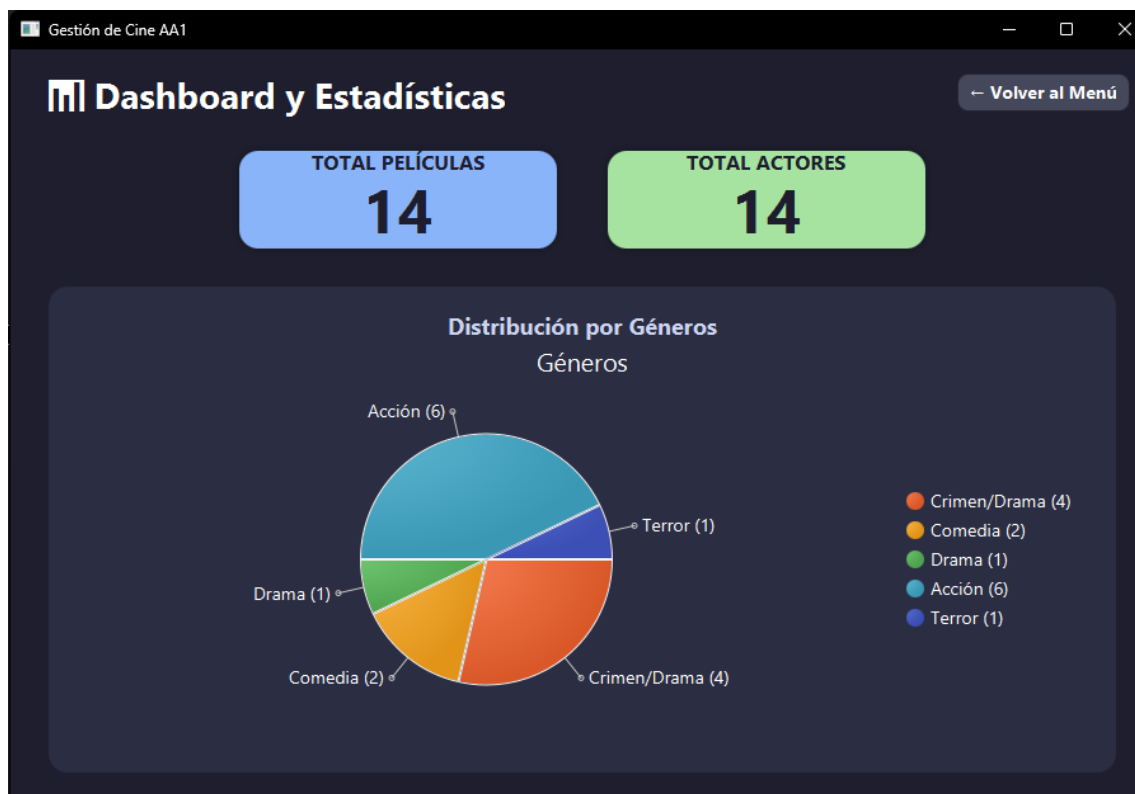
- Nombre:** Ej: Christopher Nolan
- Fecha Nac.:** [Calendar icon]
- Nacionalidad:** Ej: Británico
- Web Oficial:** www.nolan.com

**Buttons:** "Nuevo / Li...", "Eliminar", "GUARDAR DIRECTOR", "Exportar a CSV"

## Dashboard de Estadísticas (Corregido)

He reactivado la pantalla de Dashboard para que muestre datos reales y útiles, conectándola con la base de datos a través de consultas de agrupación (GROUP BY).

- **Gráfico Dinámico:** El gráfico de sectores ("quesitos") ahora traduce en tiempo real los IDs numéricos de la base de datos a los nombres de los géneros (Acción, Drama, etc.), ofreciendo una visión visual y veraz del contenido de la biblioteca.



## Conclusión

Conclusión y Repositorio Tras estos días de trabajo intensivo, el proyecto cuenta ahora con una base de datos robusta y normalizada, y una aplicación JavaFX estable que gestiona todas las entidades sin errores de ejecución. He aprendido la importancia de alinear el diseño lógico con la implementación final para evitar problemas de integridad.

El código fuente completo y actualizado está disponible en el repositorio:

 <https://github.com/8sandro8/Gestion-Peliculas-AA1.git>